FounderOS

AI Venture Studio — Multi-Agent Society

Build Blueprint & Scaffolding Guide
(Start → Finish)

🏆 **Track 3: Agent Society**

Powered by QwenCloud APIs · FastAPI · Next.js · LangGraph
Scaffolding-first build — API keys plugged in later

Version 1.0

1. Executive Overview

> FounderOS is a **society of specialized AI agents** that turns a founder's profile (skills, budget, time, goals) into a validated startup idea and a complete execution plan. Agents **divide the task, debate, resolve conflicts, and reach consensus** — exactly the collaboration pattern Track 3 asks for.

The transformation we deliver:

From
"I want to earn side income."

To
"Here is a validated startup, a Lean Canvas, a go-to-market plan, and a 30-day roadmap — pressure-tested by 7 specialist agents."

Why this fits Track 3: Agent Society

Track 3 Requirement	How FounderOS satisfies it
Task decomposition & role assignment	An Orchestrator decomposes "evaluate a venture" into discovery, market, finance, growth, risk, and fit sub-tasks, each owned by a dedicated agent.
Dialogue & negotiation	The Debate Engine detects contradictions between agents and runs multi-round structured argumentation until positions converge.
Conflict resolution	A Consensus Engine + Venture Partner agent weighs rebuttals and issues a final, reconciled recommendation.
Measurable efficiency gain vs single-agent	A built-in benchmark harness compares the society against a single-LLM baseline on validity, hallucinated-cost rate, risk coverage, and plan completeness (see §8).

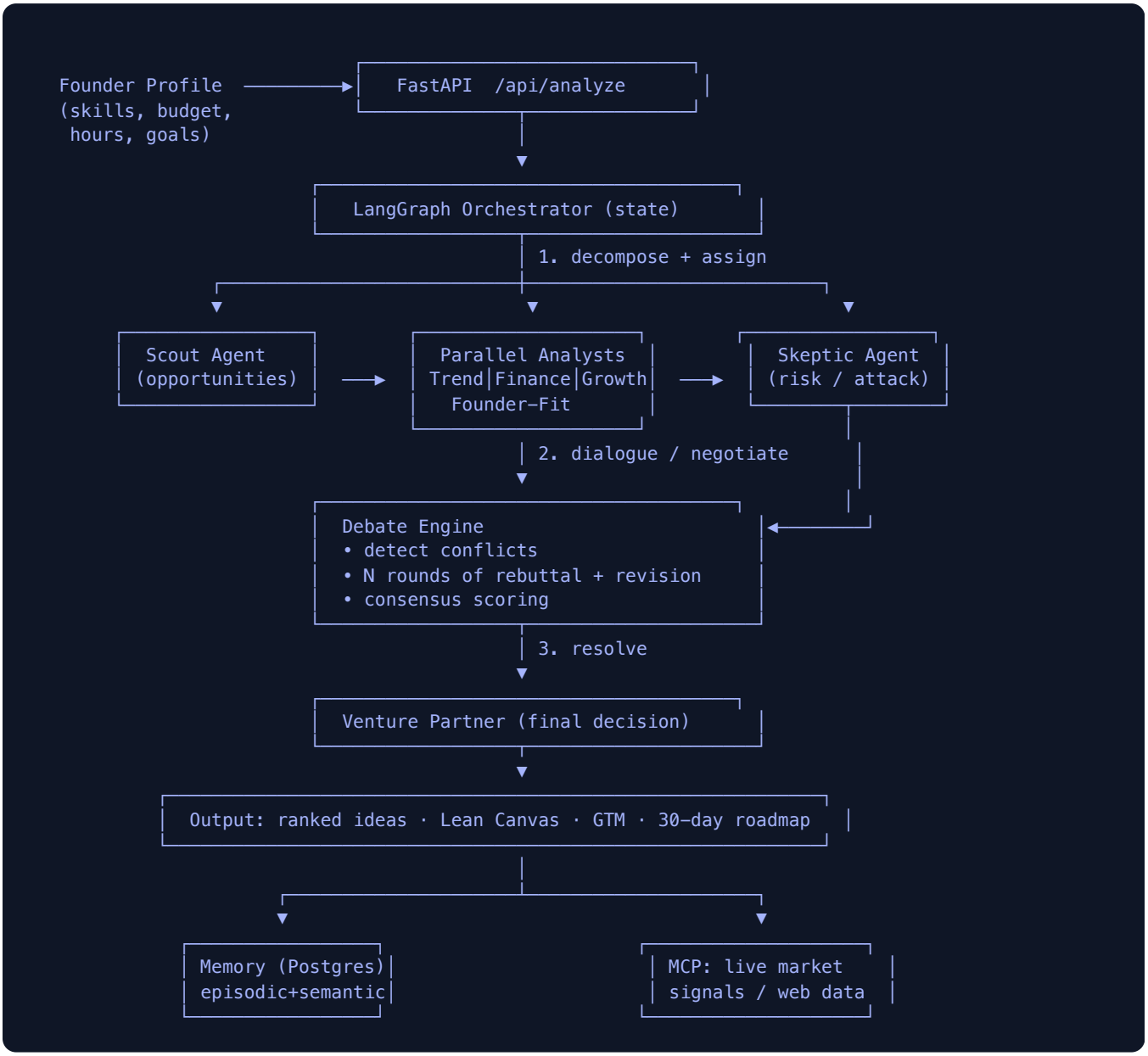
2. How the Build Targets the Judging Rubric

Every architectural decision below is chosen to score against the four rubric pillars.

Rubric Pillar (weight)	What we build to win it
Technical Depth & Engineering (30%)	• Sophisticated **QwenCloud API** usage: multi-agent prompting, JSON-mode structured output, function/tool calling, and an **MCP integration** for live market signals. • **Custom components:** a Conflict-Detection algorithm, a multi-round Debate Engine, and a Consensus scorer. • **Performance optimization:** parallel agent fan-out + response caching.
Innovation & AI Creativity (30%)	• Clean, **modular agent abstraction** (`BaseAgent`) → trivially extensible. • **LangGraph state machine** for orchestration (advanced pattern). • Robust **error handling & graceful degradation** (mock mode without keys). • Non-trivial logic: agents revise positions across rounds — emergent, not scripted.

Problem Value & Impact (25%)	• Authentic pain point: students/young founders lack validation skills & structured plans. • **Productizable**: clear SaaS path, B2B2C (incubators, universities). • **Open-source friendly**: pluggable agents, model-agnostic provider layer.
Presentation & Documentation (15%)	• Live UI **visualizes the debate** in real time (agent cards, conflict badges, consensus). • This blueprint + architecture docs + a 3–5 min demo script. • Benchmark dashboard makes the efficiency gain **visible and measurable**.

3. System Architecture



Tech Stack (scaffolded; keys added later)

Layer	Technology	Status in scaffold
Frontend	Next.js 14 + TailwindCSS	SCAFFOLDED
Backend API	FastAPI (Python 3.10+)	SCAFFOLDED
Orchestration	LangGraph state machine	TO WIRE
AI Layer	**QwenCloud API** (OpenAI-compatible SDK), model `qwen-plus`	KEY LATER
Tooling	MCP server for market/web signals	KEY LATER
Memory	PostgreSQL (episodic) + vector store (semantic)	TO WIRE

4. The Agent Society — Roles & Task Division

Seven agents, each a single responsibility. This is the *task decomposition* Track 3 rewards.

Agent	Owned Sub-Task	Structured Output
Opportunity Scout	Discover gaps, generate hypotheses	5 opportunity candidates
Trend Analyst	Demand & market growth (uses MCP signals)	Market attractiveness score
Founder-Fit	Skill match + execution likelihood	Founder-fit score
Finance	Cost, revenue, profitability	Financial feasibility score
Growth	Acquisition channels, GTM	Go-to-market plan
Skeptic	Attack assumptions, surface failure modes	Risk report
Venture Partner	Facilitate consensus, final call	Investment-style memo + execution plan

Conflict Resolution Protocol (the core innovation)

1. **Independent analysis** — agents produce positions without seeing each other.
2. **Conflict detection** — the engine flags contradictions (e.g. Growth says "scale paid ads" vs Finance "budget supports 0 ad spend").
3. **Debate rounds** — conflicting agents exchange rebuttals; each may revise its stance (up to N rounds).
4. **Consensus scoring** — quantify agreement; unresolved tensions are surfaced, not hidden.
5. **Final reconciliation** — Venture Partner issues the decision, weighting the Skeptic explicitly.

5. Scaffolding-First Principles (API keys later)

> The entire system is built to **run end-to-end before any real API key exists**. Keys are injected purely through environment variables, and every external call has a deterministic **mock fallback**.

5.1 Single config surface — `backend/config.py`

```python
# All secrets read from env. Empty by default → app still boots.
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    # === QwenCloud (OpenAI-compatible) — fill later ===
    qwen_api_key: str = ""            # <-- paste key in .env when ready
    qwen_base_url: str = "https://dashscope-intl.aliyuncs.com/compatible-mode/v1"
    qwen_model: str = "qwen-plus"

    # === MCP tool server — fill later ===
    mcp_server_url: str = ""

    # === Memory ===
    database_url: str = "postgresql+asyncpg://postgres:password@localhost:5432/founderos"

    # === Mode flags ===
    use_mock_llm: bool = True   # auto-False once qwen_api_key is set

    class Config:
        env_file = ".env"
```

5.2 Provider layer with mock fallback — `backend/llm/provider.py`

```python
from openai import OpenAI
from ..config import settings

class QwenProvider:
    def __init__(self):
        self.mock = not settings.qwen_api_key      # no key → mock mode
        if not self.mock:
            self.client = OpenAI(
                api_key=settings.qwen_api_key,
                base_url=settings.qwen_base_url,   # OpenAI-compatible endpoint
            )

    def chat(self, system: str, user: str, max_tokens: int = 2000) -> str:
        if self.mock:
            return _mock_response(system, user)    # deterministic fixture
        resp = self.client.chat.completions.create(
            model=settings.qwen_model,
            response_format={"type": "json_object"},  # structured output
            messages=[{"role":"system","content":system},
                      {"role":"user","content":user}],
            max_tokens=max_tokens,
```

```
        )
        return resp.choices[0].message.content
```

Result: `uvicorn` + `npm run dev` work immediately with mock data. The day a QwenCloud key lands, you paste it into `.env`, set `USE_MOCK_LLM=false`, and the same code path goes live — zero refactor.

5.3 The `.env.example` you ship (no secrets committed)

```
# Copy to .env and fill when keys are available
QWEN_API_KEY=                          # <-- paste later
QWEN_BASE_URL=https://dashscope-intl.aliyuncs.com/compatible-mode/v1
QWEN_MODEL=qwen-plus
MCP_SERVER_URL=                        # <-- paste later
DATABASE_URL=postgresql+asyncpg://postgres:password@localhost:5432/founderos
USE_MOCK_LLM=true                        # flip to false once key is set
CORS_ORIGINS=http://localhost:3000
```

6. The Build, Phase by Phase (Start → Finish)

Phase 0 Project Scaffold & Tooling

- Repo layout: `backend/` (FastAPI), `frontend/` (Next.js), `docs/`.
- `requirements.txt`: fastapi, uvicorn, pydantic, **openai** (Qwen-compatible), langgraph, sqlalchemy, asyncpg, pytest.
- Commit `.env.example`, add `.env` to `.gitignore`.
- **Deliverable:** `uvicorn backend.main:app` returns a health check; `npm run dev` renders a shell.

Phase 1 Provider & Mock Layer SCAFFOLDING CORE

- Build `QwenProvider` with mock fallback (§5.2) and JSON-mode parsing utilities.
- Author deterministic mock fixtures for every agent so the full pipeline runs key-free.
- **Deliverable:** end-to-end `/api/analyze` returns a complete (mocked) recommendation.

Phase 2 Agent Abstraction & the Seven Agents

- `BaseAgent`: shared provider, profile formatter, JSON parsing, retry-on-malformed-JSON.
- Implement Scout, Trend, Founder-Fit, Finance, Growth, Skeptic, Venture Partner — each a focused system prompt + typed output.
- **Deliverable:** each agent independently produces a validated `AgentOutput`.

Phase 3 LangGraph Orchestration (task division)

- Define a typed graph state; nodes = agents, edges = data flow.
- Fan-out the four analysts in **parallel** (performance optimization), then converge.
- **Deliverable:** orchestrator runs Scout → parallel analysts → Skeptic deterministically.

Phase 4 Debate & Consensus Engine (negotiation)

- Conflict detector → N debate rounds → consensus scorer (the custom algorithm judges reward).
- Persist every round so the UI can replay the negotiation.
- **Deliverable:** measurable consensus score; unresolved conflicts surfaced.

Phase 5 Memory Loop (episodic + semantic)

- Wire PostgreSQL episodic store; feed prior outcomes back as `memory_context` on re-runs.
- Semantic layer: learned insights (e.g. "this founder executes B2B better than B2C").
- **Deliverable:** second run for the same user demonstrably improves over the first.

Phase 6 MCP Integration (live signals)

- Connect an MCP server so Trend Analyst pulls real market/web signals via tool calls.
- Mock the tool responses until the MCP endpoint key is supplied.
- **Deliverable:** Trend analysis cites live data when MCP is configured.

Phase 7 Frontend — Visualize the Society

- Four-phase UI: Profile → Analyzing → Debating → Results.
- Agent cards, live conflict badges, consensus meter, execution-plan renderer.
- **Deliverable:** the debate is visible — directly scores the Presentation pillar.

Phase 8 Benchmark Harness (efficiency gain)

- Single-agent baseline vs full society on the same profiles; log the metrics in §8.
- **Deliverable:** a chart proving the society beats one agent — Track 3's explicit ask.

Phase 9 Go Live: Insert Keys

- Paste `QWEN_API_KEY` + `MCP_SERVER_URL` into `.env` ; set `USE_MOCK_LLM=false` .
- Run benchmark again with the real model; record final numbers for the demo.
- **Deliverable:** production pipeline on real QwenCloud inference.

7. QwenCloud & MCP — Technical Depth Details

7.1 Sophisticated QwenCloud usage

- **OpenAI-compatible SDK** against the DashScope endpoint — one client, seven agents.
- **JSON mode** (`response_format=json_object`) for reliable structured outputs.
- **Tool / function calling** in the Trend Analyst to invoke MCP market lookups.
- **Model tiering**: `qwen-plus` for reasoning-heavy agents, `qwen-turbo` for cheap fan-out — a cost/perf optimization.
- **Response caching** keyed on (agent, profile-hash) to cut latency & spend on repeat runs.

7.2 MCP integration sketch

```
# Trend Analyst declares a tool; Qwen decides when to call it.
tools = [{
  "type": "function",
  "function": {
    "name": "get_market_signals",
    "description": "Fetch search-trend & demand data for a niche",
    "parameters": {"type":"object",
      "properties": {"niche": {"type":"string"}}}
  }
}]
# tool call is routed to the MCP server at settings.mcp_server_url
# → mocked deterministically while MCP_SERVER_URL is empty
```

8. Measurable Efficiency Gain vs Single-Agent Baseline

Track 3 requires a *measurable* advantage. We run identical founder profiles through both systems and report:

Metric	Single-Agent Baseline	Agent Society (target)	Why it improves
Plan completeness (0–100)	~55	**90+**	Dedicated agents cover blind spots.
Hallucinated-cost rate	High	**Low**	Finance + Skeptic cross-check numbers.
Risk coverage (# real risks found)	1–2	**6–8**	Skeptic dedicated to attack.
Founder-fit accuracy	Generic	**Tailored**	Founder-Fit agent + memory.
Self-contradiction rate	Unchecked	**Resolved**	Debate engine reconciles conflicts.

Instrumentation also logs consensus score, agent-agreement rate, and per-stage latency for the demo dashboard.

9. Final Repository Layout

```
founderos/
├── frontend/                 # Next.js + Tailwind — visualizes the society
│   └── src/{app, components}/  # ProfileForm, AgentDebate, StartupCard, ExecutionPlan
├── backend/
│   ├── config.py              # single env surface (keys later)
│   ├── llm/provider.py        # QwenProvider + mock fallback
│   ├── agents/                # base + 7 specialized agents
│   ├── consensus/debate_engine.py
│   ├── memory/{episodic,semantic}.py
│   ├── mcp/client.py          # MCP tool routing
│   ├── graph.py               # LangGraph orchestrator
│   ├── benchmark/baseline.py  # single-agent vs society
│   ├── models.py · main.py
├── docs/{blueprint, architecture, demo_script}
└── .env.example               # placeholders — no secrets
```

10. Quickstart (runs key-free in mock mode)

```
# 1. Backend
cd backend && python -m venv venv && source venv/bin/activate
pip install -r requirements.txt
cp ../.env.example ../.env        # leave keys blank for now
uvicorn backend.main:app --reload --port 8000

# 2. Frontend
cd frontend && npm install && npm run dev    # http://localhost:3000

# 3. When QwenCloud key is ready:
#    edit .env → QWEN_API_KEY=... and USE_MOCK_LLM=false → restart. Done.
```

One-line summary for judges: FounderOS is a 7-agent society on QwenCloud that decomposes venture evaluation, debates to consensus, and provably out-performs a single agent — built scaffold-first so it runs today and goes live the moment a key is added.